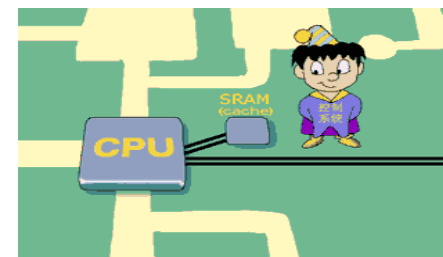
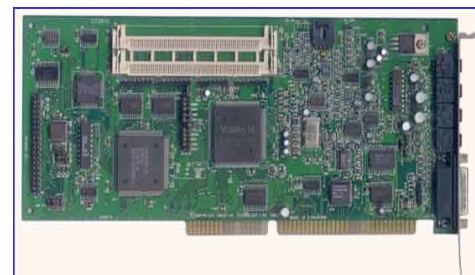
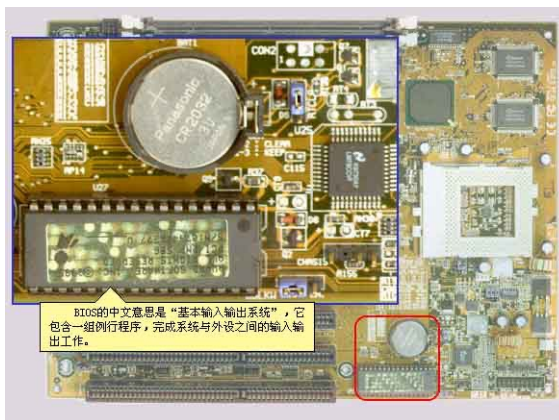




计算机组成原理 (计算机系统基础II)





计算机系统层次结构

程序执行结果

不仅取决于

算法、程序设计

而且取决于

语言处理系统

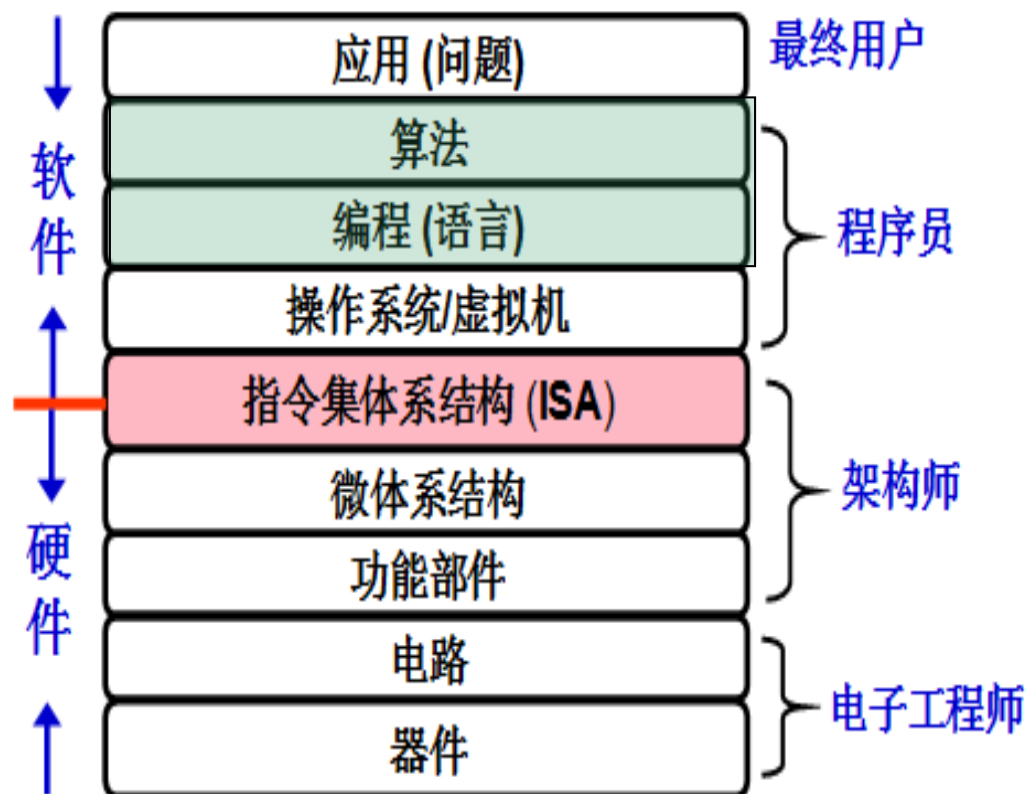
操作系统

指令系统(ISA)

微体系结构

必须将各层
次关联起来
解决问题

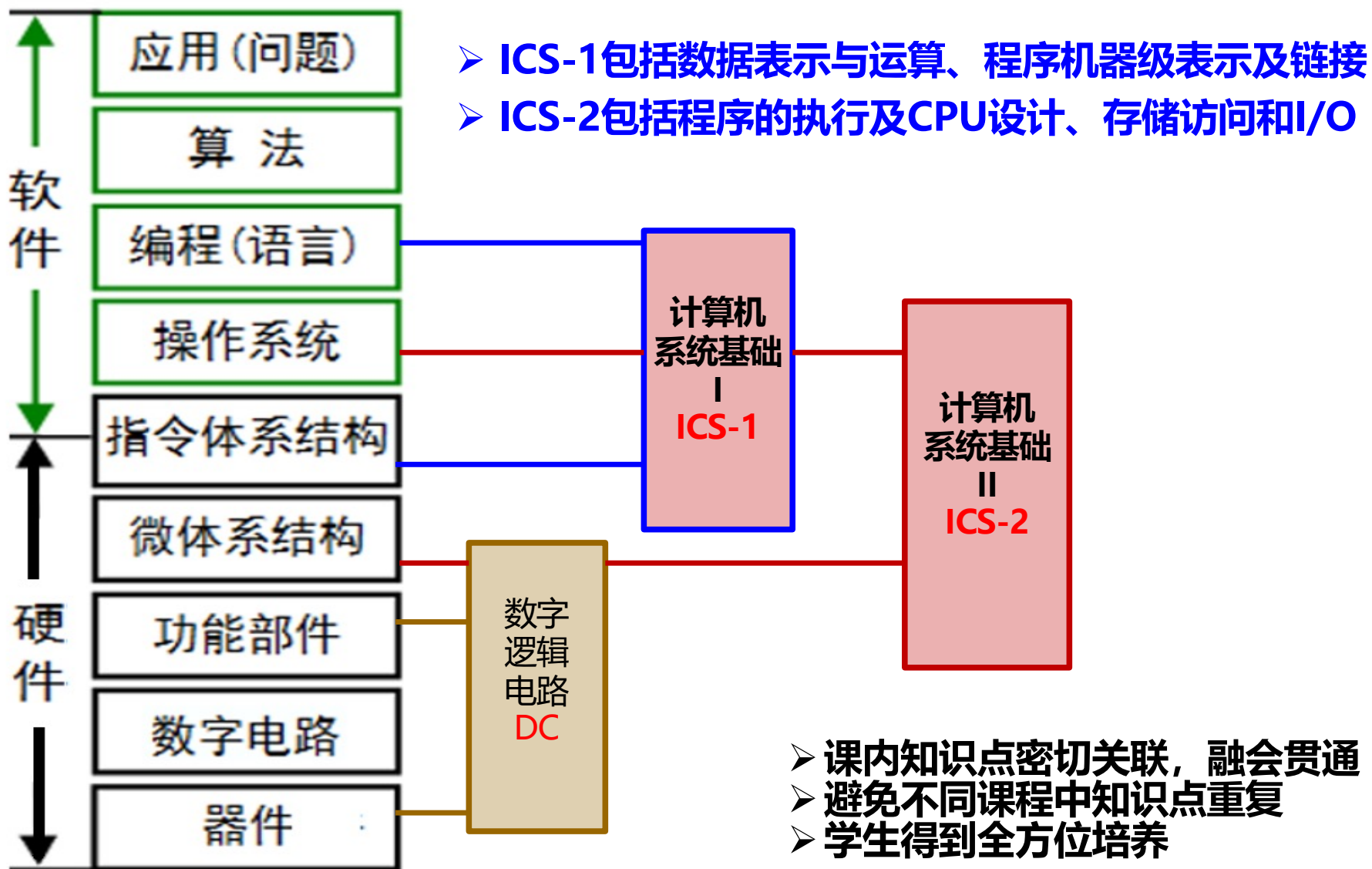
计算机系统抽象层的转换





课程介绍

计算机组成原理





计算机系统基础I

计算机系统基础II

模块1：信息的表示和处理

- 二进制与数字电路
- 整数的表示
- 浮点数的表示
- 信息存储与位操作
- 算术运算

模块2：程序的机器级表示

- 程序转换概述
- 指令系统概述
- 访存和算术逻辑运算
- 顺序控制
- 过程调用
- 复杂数据类型
- 缓冲区溢出

模块3：链接、加载和执行

- 编译器驱动程序
- 目标文件
- 符号解析和重定位
- 动态链接和库打桩



模块1：指令系统

- 指令集体系结构
- **RISC CISC**
- 指令系统设计
- 主流指令集介绍

模块2：运算方法和部件

- 定点加减运算
- 定点乘法运算
- 定点除法运算
- 运算器设计

模块3：处理器体系结构

- 组合逻辑电路和时序逻辑电路
- 单周期**CPU**设计
- 流水线原理
- 流水线**CPU**设计

模块4：存储器层次结构

- 半导体存储器
- 硬盘存储器
- 程序访问的局部性
- 高速缓存与程序性能

模块5：虚拟存储器

- 虚拟存储器概述
- 虚拟存储器实现
- 存储系统实例
- 堆区动态分配

模块6：进程与异常控制流

- 进程与进程的控制
- 异常和中断的处理
- 信号与非本地跳转

模块7：系统级输入/输出

- 用户空间I/O软件
- I/O硬件和I/O方式
- 内核空间I/O软件概述

第一章 指令系统

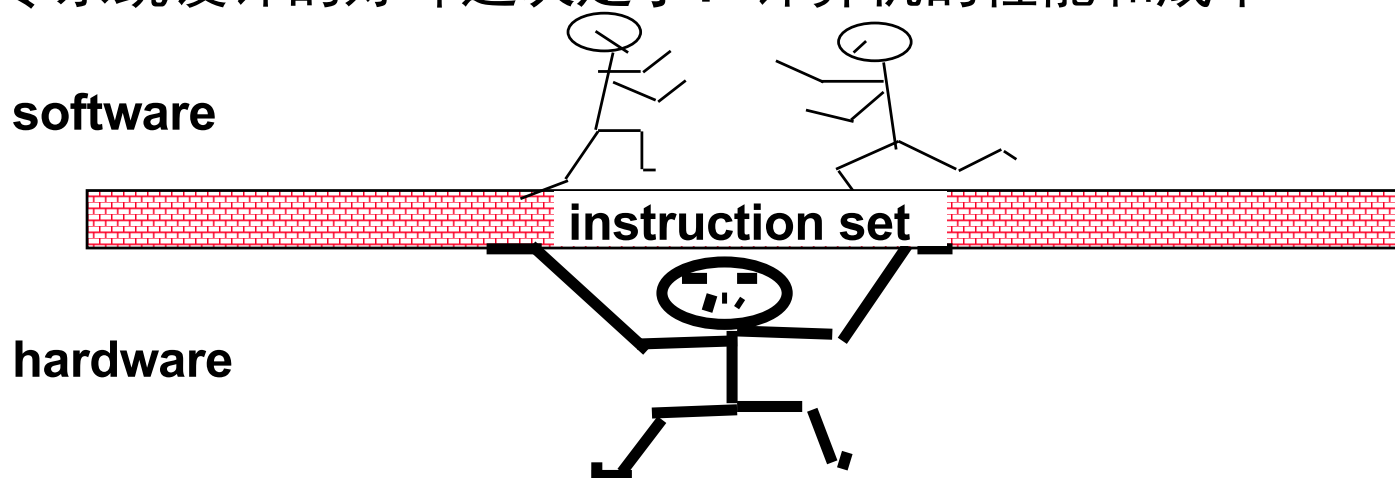
指令系统概述
典型的指令系统
指令系统设计



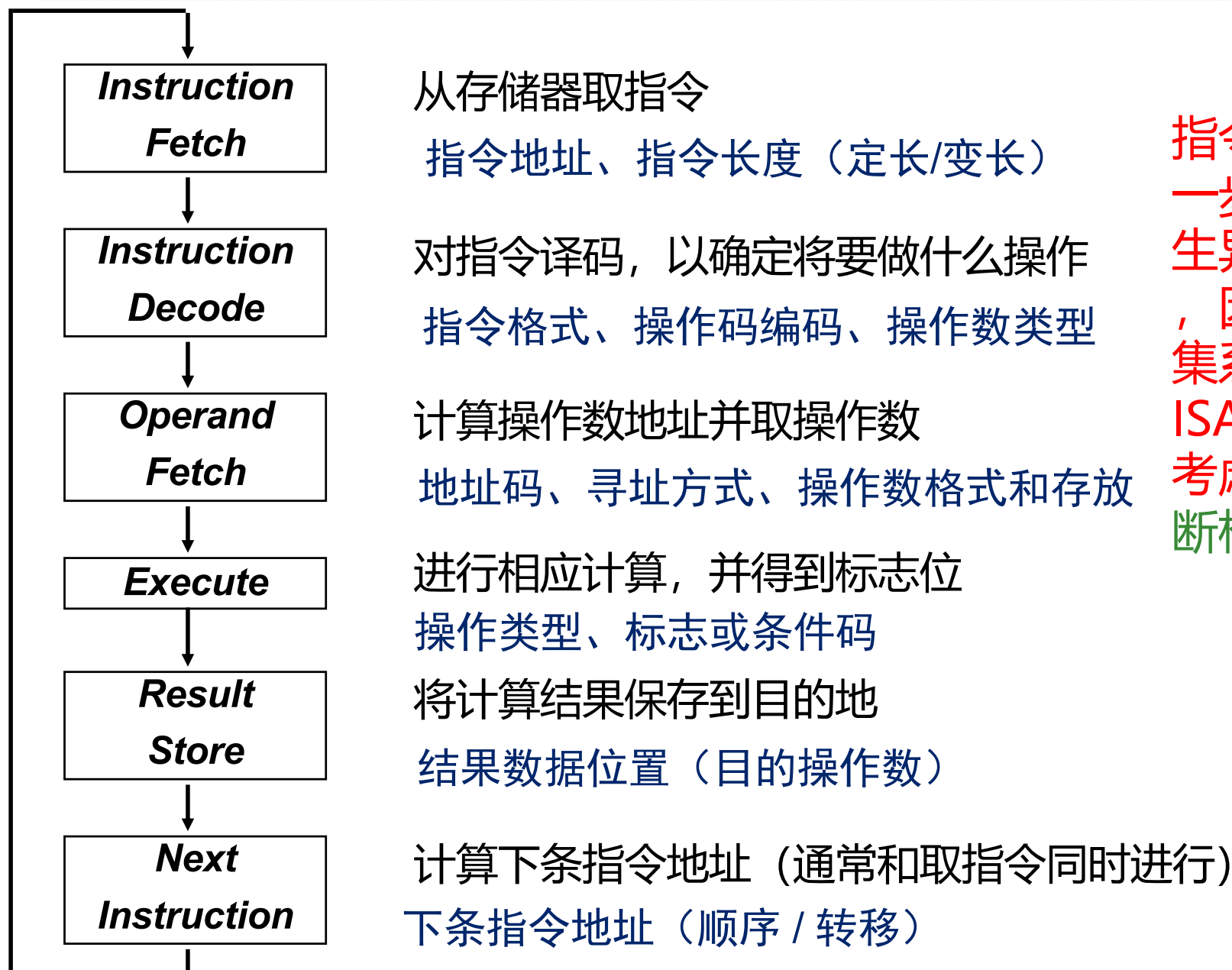
指令系统概述

计算机组成原理

- 一台计算机中所有机器指令的集合称为该机的指令系统或指令集
- 指令系统处在软/硬件交界面，同时被硬件设计者和系统程序员看到
- 硬件设计者角度：指令系统为CPU提供功能需求，要求易于硬件设计
- 系统程序员角度：通过指令系统来使用硬件，要求易于编写编译器
- 指令系统设计的好坏还决定了：计算机的性能和成本



指令系统是表征计算机性能的重要因素，它的格式与功能不仅直接影响到机器的硬件结构，而且也直接影响到系统软件，影响到机器的适用范围。



指令执行的每一步都可能发生异常或中断，因此，指令集系统架构（ISA）还需要考虑异常和中断机制



指令格式的选择应遵循的几条基本原则

- 应尽量短
- 要有足够的操作码位数
- 指令编码必须有唯一的解释，否则机器不知道该如何执行指令
- 指令字长应是字节的整数倍
- 合理地选择地址字段的个数
- 指令尽量规整

一般通过对操作码进行不同的编码来定义不同的含义，操作码相同时，再由功能码定义不同的含义！

与指令集设计相关的重要方面

- 操作码的全部组成：操作码个数 / 种类 / 复杂度
- 数据类型：对哪几种数据类型完成操作
- 指令格式：指令长度 / 地址码个数 / 各字段长度
- 通用寄存器：个数 / 功能 / 长度
- 寻址方式：操作数地址的指定方式
- 下条指令的地址如何确定：顺序，PC+1；条件转移；无条件转移；.....
- 异常和中断机制，包括存储保护方式等



一个完善的指令系统应满足以下四方面的要求：

完备性

用汇编语言编写各种程序时，指令系统直接提供的指令足够使用，而不必用软件来实现。要求指令系统丰富、功能齐全、使用方便。

一台计算机中最基本、必不可少的指令是不多的。许多指令可用最基本的指令编程来实现。例如，乘除运算指令、浮点运算指令可直接用硬件来实现，也可用基本指令编写的程序来实现。采用硬件指令的目的是提高程序执行速度，便于用户编写程序。

有效性

利用该指令系统所编写的程序能够高效率地运行。高效率主要表现在程序占据存储空间小、执行速度快。

一般来说，一个功能更强、更完善的指令系统，必定有更好的有效性。



规整性

包括指令系统的对称性、匀齐性、指令格式和数据格式的一致性。

对称性：在指令系统中所有的寄存器和存储器单元都可同等对待，所有的指令都可使用各种寻址方式；

匀齐性：一种操作性质的指令可以支持各种数据类型，如算术运算指令可支持字节、字、双字整数的运算，十进制数运算和单、双精度浮点数运算等；

指令格式和数据格式的一致性：指令长度和数据长度有一定的关系，以方便处理和存取。例如指令长度和数据长度通常是字节长度的整数倍。

兼容性

系列机各机种之间具有相同的基本结构和共同的基本指令集，因而指令系统是兼容的，即各机种上基本软件可以通用。

但由于不同机种推出的时间不同，在结构和性能上有差异，做到所有软件都完全兼容是不可能的，只能做到“向下兼容”，即旧机器机上运行的软件仍可以在新机器上运行（但不能利用新增硬件部件和扩展指令）。



CISC vs. RISC

按指令格式的复杂度来分，有两种类型计算机：

复杂指令集计算机CISC (Complex Instruction Set Computer)

精简指令集计算机RISC (Reduce Instruction Set Computer)

早期CISC设计风格的主要特点

(1) 指令系统复杂

变长操作码 / 变长指令字 / 指令多 / 寻址方式多 / 指令格式多

(2) 指令周期长

绝大多数指令需要多个时钟周期才能完成

(3) 各种指令都能访问存储器

除了专门的存储器读写指令外，运算指令也能访问存储器

(4) 采用微程序控制

(5) 有专用寄存器

(6) 难以进行编译优化来生成高效目标代码

例如，VAX-11/780小型机

16种寻址方式；9种数据格式；303条指令；

一条指令包括1~2个字节的操作码和下续N个操作数说明符。

一个说明符的长度达1 ~10个字节。



CISC vs. RISC

◆ CISC的缺陷

- 日趋庞大的指令系统不但使计算机的研制周期变长，而且难以保证设计的正确性，难以调试和维护，并且因指令操作复杂而增加机器周期，从而降低了系统性能。

对CISC进行测试，发现：

各种指令使用频率相差悬殊：常用的较为简单的指令，仅占指令总数的20%，但在程序中使用的频率却占80%。



CISC vs. RISC

- 1975年IBM公司开始研究指令系统的合理性问题，John Cocks提出精简指令集计算机 RISC (Reduce Instruction Set Computer)。
- 1982年美国加州伯克利大学的RISC-I，斯坦福大学的MIPS，IBM公司的IBM801相继宣告完成，这些机器被称为第一代RISC机。

RISC技术的实质是要求指令系统简化，尽量使用寄存器-寄存器操作指令，指令操作在一个周期内完成，指令格式力求一致，以利于提高编译的效率。



CISC vs. RISC

RISC设计风格的主要特点

(1) 简化的指令系统

指令少 / 寻址方式少 / 指令格式少 / 指令长度一致

(2) 以RR方式工作

除Load/Store指令可访问存储器外，其余指令都只访问寄存器。

(3) 指令周期短

以流水线方式工作，因而除Load/Store指令外，其他简单指令都只需一个或一个不到的时钟周期就可完成。

(4) 采用大量通用寄存器，以减少访存次数

(5) 采用组合逻辑电路控制，不用或少用微程序控制

(6) 采用优化的编译系统，力求有效地支持高级语言程序

ARM、MIPS、RISC (RISC-I 到 RISC-V) 系列架构是典型的RISC处理器，80年代以来新的指令集大多采用RISC体系结构

x86因为“兼容”的需要，保留了CISC的风格，同时也借鉴了RISC思想



CISC vs. RISC

若设高级语言程序经编译后在机器上运行的**机器指令数为I**；每条机器指令执行时所需要的**平均机器周期数(CPI)是C**；每个机器周期的**执行时间为T**；则计算机执行程序的时间 **$P=I \times C \times T$** 。

RISC/CISC的I、C、T统计表

	I	C	T
RISC	1.2 ~ 1.4	1.3 ~ 1.7	< 1
CISC	1	4 ~ 6	1

由于RISC指令比较简单，原CISC机中比较复杂的指令可用RISC中一段子程序代替，因此，RISC中的**I**比CISC的多20%—40%；

但是，RISC的大多数指令只用一个机器周期实现，**C**的值比CISC的小得多

又因为RISC结构简单，完成一个操作所经过的数据通道较短，因此，**T**的值也更小。

下列关于RISC的叙述中，**错误**的是（ ）。

- ☒ A RISC普遍采用微程序控制器
- ☐ B RISC大多数指令在一个时钟周期内完成
- ☐ C RISC的内部通用寄存器数量相对CISC多
- ☐ D RISC的指令数、寻址方式和指令格式种类相对CISC少

提交



回顾：冯.诺依曼结构机器对指令规定：

- ◆ 用二进制表示，和数据一起存放在主存中
- ◆ 由两部分组成：操作码和操作数（或其地址码）
 - **Operation Code**: defines the operation type
 - **Operands**: indicate operation source and destination

一条指令应包含如下信息：

进行何种操作：即操作性质，体现在指令中被称为**操作码**。

操作的对象：数据来源以及如何寻找操作数。

操作结果：结果存放在何处。

下一条指令如何寻找。

后三项信息体现在指令中被统称为**地址码**，因此指令由**操作码**和**地址码**两部分组成，它的基本格式如下：

操作码字段OP	地址码字段A
---------	--------



操作码

操作码是说明指令操作性质的二进制代码，如进行加法、减法、乘法、除法、取数、存数等。

◆ 操作码的编码有两种方式

- Fixed Length Opcodes (定长操作码法)
- Expanding Opcodes (扩展操作码编法)

◆ 对不同指令集

- 代码长度更重要时：采用变长指令字、变长操作码 机器代码更紧凑
- 性能更重要时：采用定长指令字、定长操作码 便于快速访问和译码



一条指令中应该有几个地址码字段？

零地址指令

- (1) 无需操作数 如：空操作NOP / 停机HALT 等
- (2) 所需操作数为默认的 如：堆栈等

形式：

OP

一地址指令

其地址既是操作数的地址，也是结果的地址

- (1) 单目运算：如：取反 / 取负等
- (2) 双目运算：另一操作数为默认的 如：累加器等

形式：

OP	A1
----	----

二地址指令（最常用）

分别存放双目运算中两个操作数，并将其中一个地址作为结果的地址。

形式：

OP	A1	A2
----	----	----

三地址指令（RISC风格）

分别作为双目运算中两个源操作数的地址和一个结果的地址。

形式：

OP	A1	A2	A3
----	----	----	----

多地址指令

用于成批数据处理的指令，如：向量 / 矩阵等运算的SIMD指令。



基本思想

指令的操作码部分采用固定长度的编码

如：假设操作码固定为6位，则系统最多可表示64种指令

特点

译码方便，但有信息冗余

举例

IBM360/370采用：

8位定长操作码，最多可有256条指令

只提供了183条指令，有73种编码为冗余信息

机器字长32位，按字节编址

有16个32位通用寄存器，基址器B和变址器X可用其中任意一个

问题：通用寄存器编号有几位？ B和X的编号占几位？ 都是4位

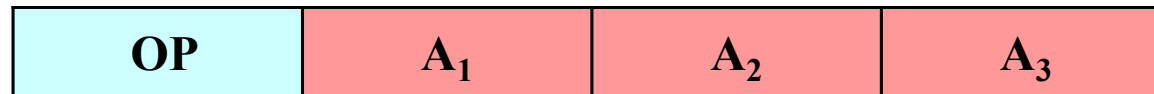


基本思想

将操作码的编码长度分成几种固定长的格式。被大多数指令集采用。

PDP-11是典型的变长操作码机器。

例：



一个16位字长的指令码，包括4位基本操作码字段和三个4位长的地址字段。4位基本操作码，若全部用于三地址指令，则有16条。显然，4位基本操作码是不够的，必须向地址码字段扩展操作码的长度。

其扩展方法及步骤如下：



1) 15条三地址指令的操作码由4位基本操作码**0000~1110**所给定，剩下一个**1111**则用于把操作码扩展到A₁地址码字段，即由4位扩展到8位；

0000	A ₁	A ₂	A ₃
0001	A ₁	A ₂	A ₃
...	...	...	...
1110	A ₁	A ₂	A ₃

15条三地址指令

2) 14条二地址指令的操作码由8位操作码的**1111,0000~1111,1101**给定，剩下的**1111,1110**和**1111,1111**又可用于把操作码扩充到A₂地址字段，即从8位又扩充到12位；

1111	0000	A ₂	A ₃
1111	0001	A ₂	A ₃
...	...	...	...
1111	1101	A ₂	A ₃

14条二地址指令



3) 31 条一地址指令的操作码由 12 位操作码的 1111,1110,0000 ~ 1111,1111,1110 给定。

0000	A ₁	A ₂	A ₃
0001	A ₁	A ₂	A ₃
...	...	...	...
1110	A ₁	A ₂	A ₃

15条三地址指令

1111	0000	A ₂	A ₃
1111	0001	A ₂	A ₃
...	...	...	...
1111	1101	A ₂	A ₃

14条二地址指令

1111	1110	0000	A ₃
...	...	...	...
1111	1110	1111	A ₃
1111	1111	0000	A ₃
...	...	...	...
1111	1111	1110	A ₃

31条一地址指令



4) 由此类推，还可以把剩下的**1111,1111,1111**扩充到A₃地址码字段而形成16位操作码，这时还可由**1111,1111,1111,0000~1111,1111,1111,1111**给出16条零地址指令。

1111	1110	0000	A ₃
...	...	...	...
1111	1110	1111	A ₃
1111	1111	0000	A ₃
...	...	...	...
1111	1111	1110	A ₃

31条一地址指令

1111	1111	1111	0000
1111	1111	1111	0001
...	...	...	...
1111	1111	1111	1111

16条零地址指令

采用这种方法，指令系统中共有15+14+31+16=76条指令。

除了这种安排外，还有多种其它安排方法。

实际的机器可以采用各种灵活的扩展方式，其思路与此类似。



上述方法是在不增加指令长度的情况下，采用对地址少的指令使用较长的操作码，对地址数多的指令使用较短的操作码。其实质是增加了指令的数量，丰富了指令的功能。

在可变长操作码的指令系统设计中，究竟使用何种扩展方法为好？

指令的**使用频度**（即在程序中出现的概率）是非常重要的依据。即**频度高的指令应分配短的操作码，频度低的指令则分配较长的操作码**。这样，既可有效地缩短操作码在程序中的平均长度，节省存储空间，又可缩短常用指令的译码时间以提高程序的运行速度。

其**缺点**是译码系统比固定操作码复杂，增加了设计控制器的难度，需要更多的硬件作支持。



例：设某指令系统指令字是16位，每个地址码为6位。若二地址指令15条，一地址指令34条，则剩下零地址指令最多有多少条？

解：

操作码按短到长进行扩展编码， $16/6=2...4$ 最多二地址

- 二地址指令， $?+6+6$ ，操作码4位，最多16种组合

15条: 0000 ~ 1110

- 一地址指令， $?+6$ ，操作码10位，开头1111，1111 000000 ~ X

$34条 = X - 1111\ 000000 + 1$ ， $X - 1111\ 000000 = 33 = 32 + 1$ ，

$X = 1111\ 100001$

- 零地址指令， $16+0$ ，操作码16位， $X+1 = 1111\ 1000\ 10$

$max = 1111\ 1111\ 1111\ 1111$ ，

$min = X + 1$ 补0 = 1111 1000 1000 0000

零地址指令最多有 $max - min + 1 = 111\ 0111\ 1111 + 1 = 111\ 1000\ 0000 = 15 * 2^7$



Addressing Modes (寻址方式)

计算机组成原理

- 什么是“寻址方式”？

指令或操作数地址的指定方式。即：根据地址找到指令或操作数的方法。

- 地址码编码由操作数的寻址方式决定

- 地址码编码原则：

指令地址码尽量短	——为什么?——>	目标代码短，省空间
操作数存放位置灵活，空间应尽量大	——>	利于编译器优化产生高效代码
地址计算过程尽量简单	——>	指令执行快

◆ 指令的寻址----简单

正常：PC增值

跳转 (jump / branch / call / return)：同操作数的寻址

◆ 操作数的寻址----复杂

操作数来源：寄存器 / 主(虚)存 / 栈顶 / 外设端口

操作数结构：位 / 字节 / 半字 / 字 / 双字 / 一维表 / 二维表 / ...

通常寻址方式指 “操作数的寻址方式”



(1) 没有专门的寻址方式位 (由操作码确定寻址方式)

如: MIPS指令, 一条指令中最多仅有一个主(虚)存地址且仅有一到两种寻址方式, Load/store型机器指令属于这种情况。

(2) 有专门的寻址方式位

如: X86指令, 一条指令中有多个操作数, 且寻址方式各不相同, 需要各自说明寻址方式, 有专门的寻址方式位。

- 有效地址的含义

操作数所在存储单元的地址 (可能是逻辑地址或物理地址)
， 可通过指令的寻址方式和地址码计算得到

- 基本寻址方式

立即 / 直接 / 间接 / 寄存器 / 寄存器间接 / 偏移 / 栈



立即寻址-寻址方式所提供的操作数直接放在指令中，紧跟在操作码的后面，与操作码一起放在代码区域中。

寄存器寻址-操作数在**CPU**的某个寄存器中

存储器操作数寻址-操作数在存储器中

Imm	$M[Imm]$	绝对寻址
(r_a)	$M[R[r_a]]$	间接寻址
$Imm(r_b)$	$M[Imm+R[r_b]]$	(基址 + 偏移量) 寻址
(r_b, r_i)	$M[R[r_b]+R[r_i]]$	变址寻址
$Imm(r_b, r_i)$	$M[Imm+R[r_b]+R[r_i]]$	变址寻址
$(, r_i, s)$	$M[R[r_i] \cdot s]$	比例变址寻址
$Imm(, r_i, s)$	$M[Imm+R[r_i] \cdot s]$	比例变址寻址
(r_b, r_i, s)	$M[R[r_b]+R[r_i] \cdot s]$	比例变址寻址
$Imm(r_b, r_i, s)$	$M[Imm+R[r_b]+R[r_i] \cdot s]$	比例变址寻址

$$\text{有效地址} = Imm + R[r_b] + R[r_i] \cdot s$$



基本寻址方式的算法和优缺点

假设: A =地址字段值, R =寄存器编号,
 EA =有效地址, (X) = X 中的内容

OP	R	A	...
----	---	---	-----

方式	算法	主要优点	主要缺点
立即数	操作数= A	指令执行速度快	操作数幅值有限
直接	$EA=A$	有效地址计算简单	地址范围有限
间接	$EA=(A)$	有效地址范围大	多次存储器访问
寄存器(直接)	操作数= (R)	指令执行快, 指令短	地址范围有限
寄存器间接	$EA=(R)$	地址范围大	额外存储器访问
偏移	$EA=A+(R)$	灵活	复杂
栈	EA =栈顶	指令短	应用有限

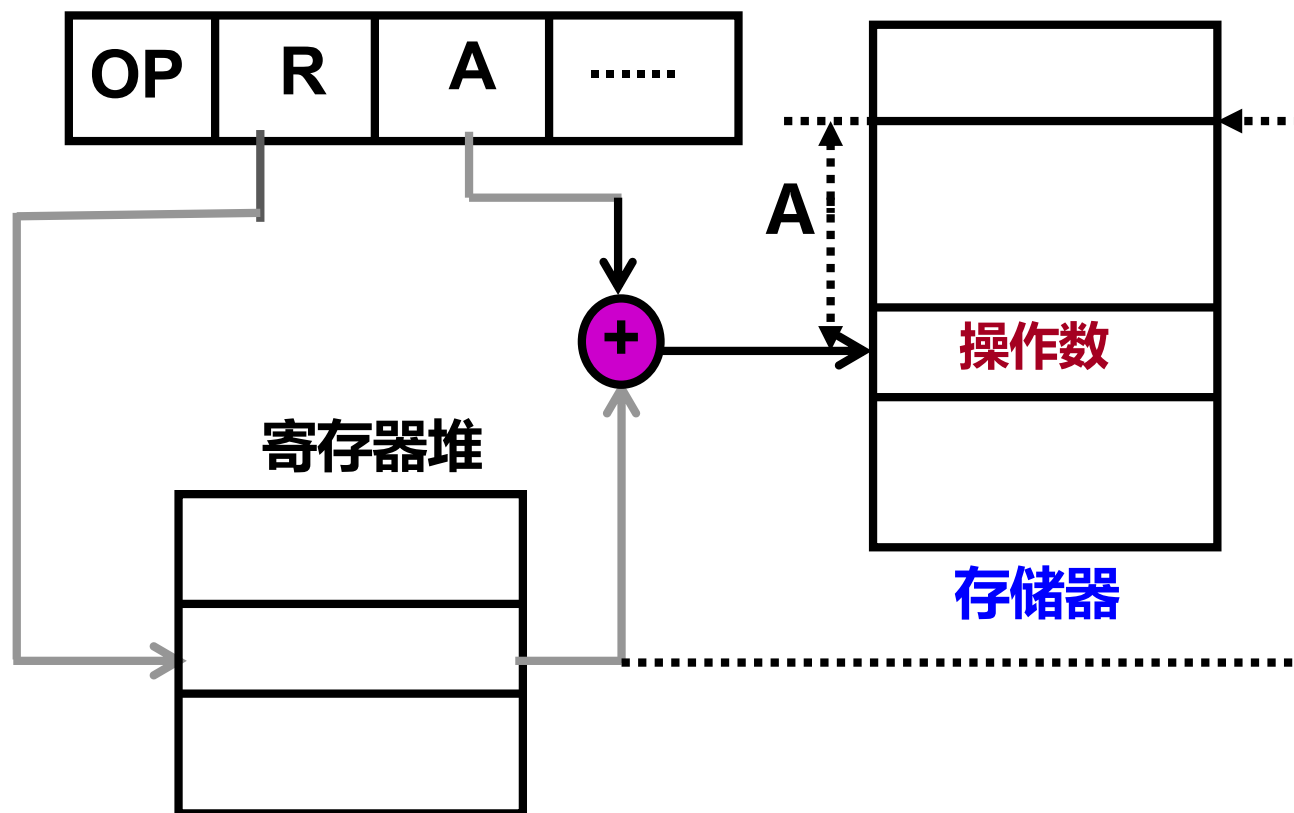
问题: 以上各种寻址方式下, 操作数在寄存器中还是在存储器中?

需要计算有效地址 EA 的操作数都是在存储器中



偏移寻址方式

指令中给出的地址码A称为形式地址



偏移寻址: $EA = (R) + A$, R可以明显给出, 也可以隐含给出

R可以是PC、基址寄存器B、变址寄存器I

- 相对寻址: $EA = (PC) + A$ 相对于当前指令处位移量为A的单元
- 基址寻址: $EA = (B) + A$ 相对于基址(B)处位移量为A的单元
- 变址寻址: $EA = (I) + A$ 相对于首址A处位移量为(I)的单元



偏移寻址方式

● 相对寻址

指令地址码给出一个偏移量(带符号数), 基准地址隐含由PC给出。

即: $EA = (PC) + A$ eg. call func, func是一个相对偏移量

可用来实现位置无关代码 (PIC) 如共享库 或 指定转移目标地址

PC (程序计数器) 一般是下条指令的地址 (x86-64, rip)

● 基址寻址

指令地址码给出一个偏移量, 基准地址明显或隐含由基址寄存器B给出。

即: $EA = (B) + A$ eg. movq 0x100(%rbx), %rax

可用来实现结构或联合体分量的访问 或 过程调用中参数的访问

● 变址寻址

指令地址码给出一个基准地址, 而偏移量(无符号数)明显或隐含由变址寄存器I给出。即: $EA = (I) + A$ eg. movq 0x100(,%rbx,8), %rax

可为循环重复操作提供一种高效机制, 如实现对线性表的方便操作



变址寻址实现线性表元素的存取

计算机组成原理

- 自动变址

指令中的地址码A给定数组首址，
变址器I每次自动加/减数组元素的
长度x。

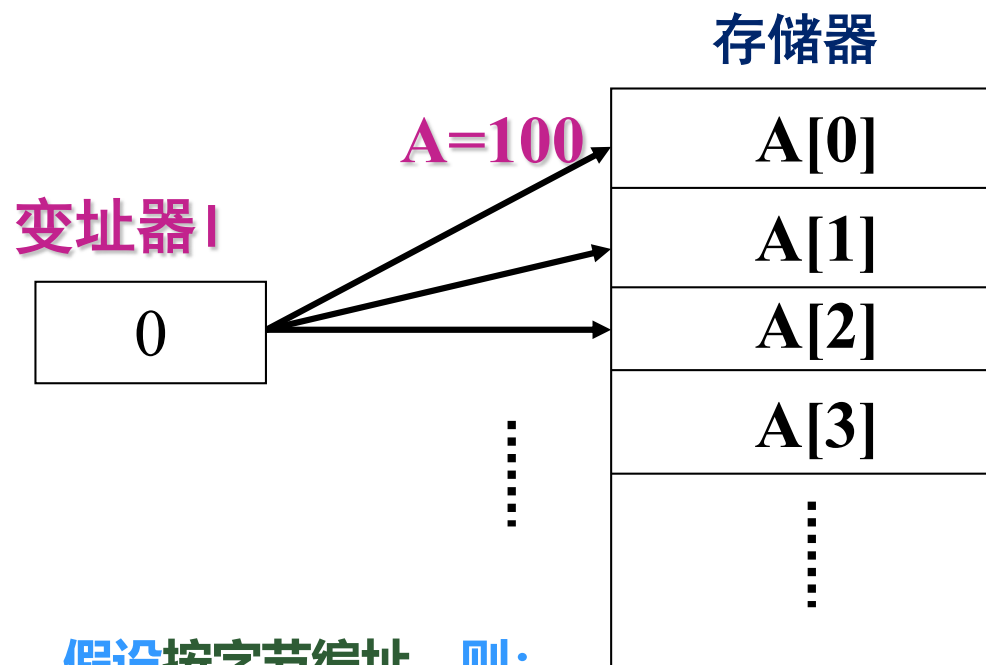
$$EA = A + (I)$$

$$I = (I) \pm x$$

例如，x86中的串操作指令

- 对于“for (i=0; i<N; i++) ...”，
即地址从低→高变化：加
- 对于“for (i=N-1; i>=0; i--) ...”，
即地址从高→低变化：减
- 可提供对线性表的方便访问

假定一维数组A从100号单元开始



假设按字节编址，则：

每个元素为1个字节时， $I = (I) \pm 1$

每个元素为4个字节时， $I = (I) \pm 4$

一般RISC机器不提供自动变址寻址，

将变址和基址寻址统一成一种偏移寻址方式



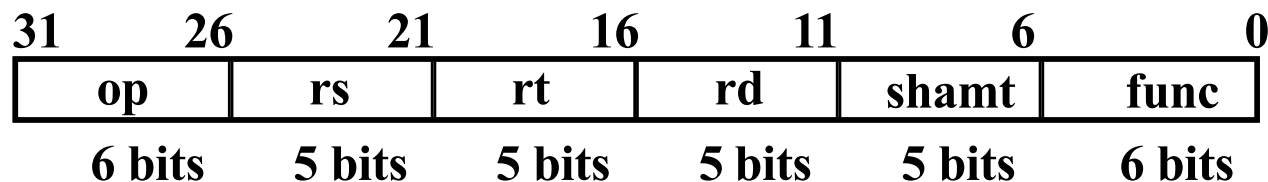
指令系统举例: MIPS32的指令格式

计算机组成原理

- ◆ 32个通用寄存器，所有指令都是32位宽，须按字地址对齐
字地址为4的倍数！

- ◆ 有三种指令格式

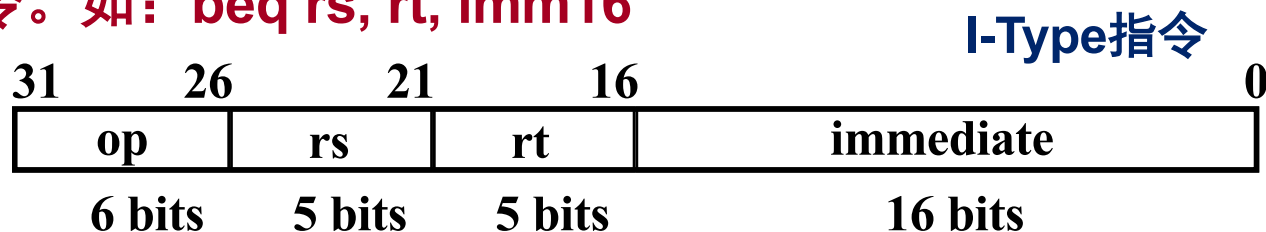
– R-Type



两个操作数和结果都在寄存器的运算指令。如: **sub rd, rs, rt**

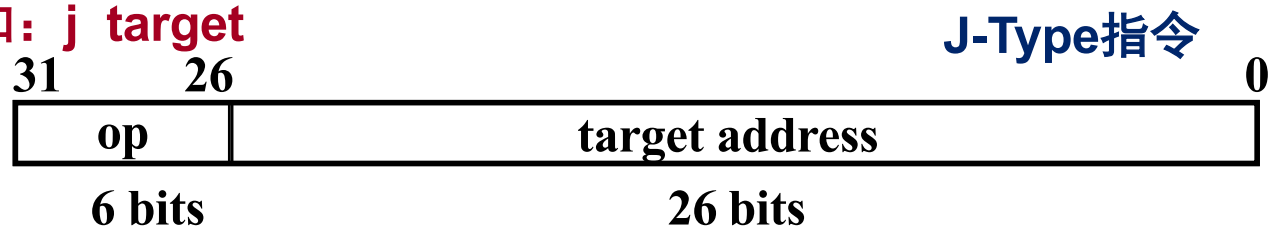
– I-Type

- 运算指令：两个寄存器、一个立即数。如: **ori rt, rs, imm16**
- LOAD和STORE指令。如: **lw rt, rs, imm16**
- 条件分支指令。如: **beq rs, rt, imm16**



– J-Type

无条件跳转指令。如: **j target**





指令系统举例：MIPS32指令字段含义

计算机组成原理

R-Type指令

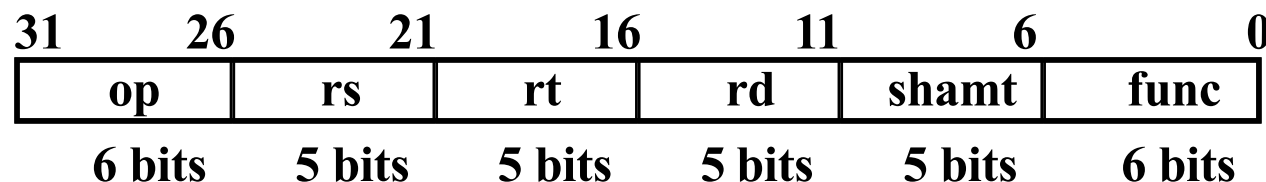
OP: 操作码

rs: 第一个源操作数寄存器

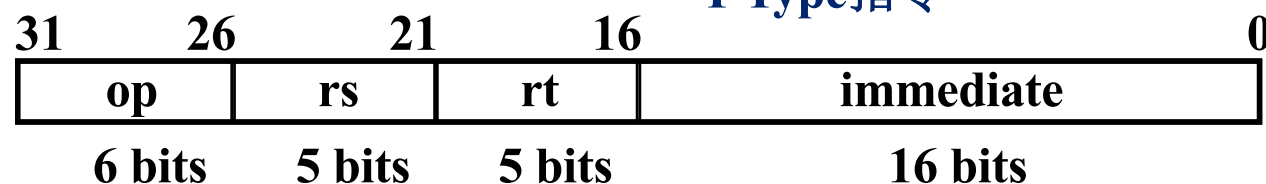
rt: 第二个源操作数寄存器

rd: 结果寄存器

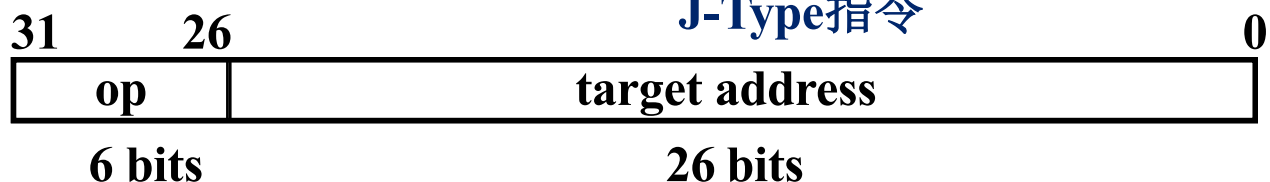
shamt: 移位指令的位移量



I-Type指令



J-Type指令



func: R-Type指令的OP字段是特定的“000000”，具体操作由func字段给定。例如：func=“100000”时，表示“加法”运算。

操作码的不同编码定义不同的含义，操作码相同时，再由功能码定义不同的含义！

immediate: 立即数或load/store指令和分支指令的偏移地址

target address: 无条件转移地址的低26位。将PC高4位拼上26位直接地址，最后添2个“0”就是32位目标地址。为何最后两位要添“0”？

指令按字地址对齐，所以每条指令的地址都是4的倍数（最后两位为0）。



指令系统举例：x86-64的指令格式

计算机组成原理

前缀：包括指令、段、操作数长度、地址长度四种类型

前缀类型：	指令前缀	段前缀	操作数长度	地址长度
字节数：	0或1	0或1	0或1	0或1

指令：含操作码、寻址方式、SIB、位移量和直接数据五部分，位移量和立即数都可是1/2/4/8B。SIB中基址B和变址I都可以是通用寄存器中任一个，SS给出比例因子。



变长指令字：1B~17B

变长操作码：4b / 5b / 6b / 7b / 8b / ...

变长操作数：Byte / Word / DW / QW

变长寄存器：8位 / 16位 / 32位 / 64位



例. 一种二地址指令的结构如下所示:

6位		4位	1位	2位	16位
OP	-	通用寄存器	I	X	偏移量D

其中I为间接寻址标志位, X为寻址模式字段, D为偏移量字段, 通过I, X, D的组合, 可构成下表所示的寻址方式。请写出6种寻址方式的名称。

寻址方式	I	X	有效地址E算法	说明
(1)	0	00	$E = D$	
(2)	0	01	$E = (PC) \pm D$	PC为程序计数器
(3)	0	10	$E = (R_2) \pm D$	R_2 为变址寄存器
(4)	1	11	$E = (R_3)$	
(5)	1	00	$E = (D)$	
(6)	0	11	$E = (R_1) \pm D$	R_1 为基址寄存器

(1) 直接寻址

(3) 变址寻址

(5) 间接寻址

(2) PC相对寻址

(4) 寄存器间接寻址

(6) 基址寻址



指令格式中的 **S(源)**，**D(目标)**都是通用寄存器，**M**是主存中的一个单元。

STO (OP)=(1B)_H, **STO**为写数指令;

LAD (OP)=(3C)_H, LAD为读数指令。





1) 分析三种指令格式与寻址方式特点。



答：MOV指令是单字长二地址指令，RR型；

STO指令是双字长二地址指令，RS型，其中目标操作数M采用基址寻址或变址寻址，源操作数S由源寄存器决定；

LAD指令也是双字长二地址指令，RS型，其中目标操作数D由目标寄存器决定，源操作数M由20位地址（直接寻址）决定。



2) CPU完成哪一种操作所花时间最短？哪一种操作花时间最长？



答：第一种指令所花时间最短，因为是RR型指令，不需要访问存储器。

第二种指令所花时间最长，因为是RS型指令，需要访问存储器，同时要寻址方式的变换运算（基址或变址），这也需要时间。

第三种指令虽然也访问存储器，但节省了求有效地址运算的时间开销。



3) 下列情况下每个十六进制指令字分别代表什么操作？其中如果有编码不正确，如何改正才能成为合法指令？

① (F0F1)_H (3CD2)_H ② (2856)_H ③ (6FD6)_H ④ (1C2)_H ⑤ (6CF1)_H (1234)_H

答：根据已知条件

MOV_(OP)=(A)_H=001010, **STO**_(OP)=(1B)_H=011011, **LAD**_(OP)=(3C)_H=111100

① (F0F1)_H (3CD2)_H = **111100**, 00,1111,0001,0011,1100,1101,0010

根据操作码111100可知该指令为LAD指令。

15	10	9	8	7	4	3	0	
OP		-		目标寄存器				LAD D, M
20位地址								

15	10	9	8	7	4	3	0	
11 1100				00	1111		0001	LAD D, M
0011 1100 1101 0010								

编码正确，其含义是把主存(13CD2)_H 地址单元的内容取至15号寄存器。



3) 下列情况下每个十六进制指令字分别代表什么操作？其中如果有编码不正确，如何改正才能成为合法指令？

① (F0F1)_H (3CD2)_H ② (2856)_H ③ (6FD6)_H ④ (1C2)_H ⑤ (6CF1)_H (1234)_H

答：根据已知条件

MOV_(OP)=(A)_H=001010, **STO**_(OP)=(1B)_H=011011, **LAD**_(OP)=(3C)_H=111100

② (2856)_H = 001010,00,0101,0110

根据操作码001010可知该指令为MOV指令。

15	10	9	8	7	4	3	0	
OP				-	目标寄存器		源寄存器	MOV D, S

15	10	9	8	7	4	3	0	
001010				00	0101	0110		MOV D, S

该指令编码正确，含义是把6号源寄存器的内容传送至5号目标寄存器。



3) 下列情况下每个十六进制指令字分别代表什么操作？其中如果有编码不正确，如何改正才能成为合法指令？

① (F0F1)_H (3CD2)_H ② (2856)_H ③ (6FD6)_H ④ (1C2)_H ⑤ (6CF1)_H (1234)_H

答：根据已知条件

MOV_(OP)=(A)_H=001010, STO_(OP)=(1B)_H=011011, LAD_(OP)=(3C)_H=111100

③ (6FD6)_H = 011011, 11, 1101, 0110

这是单字长指令，因此该指令一定是MOV指令，但操作码编码错误。

15	10	9	8	7	4	3	0	
OP			-	目标寄存器		源寄存器		MOV D, S

15	10	9	8	7	4	3	0	
001010			xx	1101		0110		MOV D, S

假设x=0，可改正为 001010, 00, 1101, 0110 = (28D6)_H



3) 下列情况下每个十六进制指令字分别代表什么操作？其中如果有编码不正确，如何改正才能成为合法指令？

① (F0F1)_H (3CD2)_H ② (2856)_H ③ (6FD6)_H ④ (1C2)_H ⑤ (6CF1)_H (1234)_H

答：根据已知条件

MOV_(OP)=(A)_H=001010, **STO**_(OP)=(1B)_H=011011, **LAD**_(OP)=(3C)_H=111100

④ (1C2)_H = 000000,01,1100,0010

这是单字长指令，因此该指令一定是MOV指令，但操作码编码错误。

15	10	9	8	7	4	3	0	
OP			-	目标寄存器		源寄存器		MOV D, S

15	10	9	8	7	4	3	0	
001010			xx	1100		0010		MOV D, S

假设x=0，可改正为 001010,00,1100,0010 = (28C2)_H



3) 下列情况下每个十六进制指令字分别代表什么操作？其中如果有编码不正确，如何改正才能成为合法指令？

① (F0F1)_H (3CD2)_H ② (2856)_H ③ (6FD6)_H ④ (1C2)_H ⑤ (6CF1)_H (1234)_H

答：根据已知条件

MOV_(OP)=(A)_H=001010, **STO**_(OP)=(1B)_H=011011, **LAD**_(OP)=(3C)_H=111100

⑤ (6CF1)_H (1234)_H = 011011, 00, 1111, 0001, 0001, 0010, 0011, 0100

根据操作码011011可知该指令为STO指令。

15	10	9	8	7	4	3	0	
OP		基址		源寄存器		变址寄存器		STO S, M
位移量								
15	10	9	8	7	4	3	0	
011011		00		1111		0001		STO S, M
1234H								

编码正确，其含义是把15号寄存器存入采用变址寻址的存储器单元中。



第一章 指令系统

指令系统概述
典型的指令系统
指令系统设计



Y86-64 指令集

计算机组成原理

Byte	0	1	2	3	4	5	6	7	8	9
halt	0	0								
nop	1	0								
cmovXX rA, rB	2	fn	rA	rB						
irmovq V, rB	3	0	F	rB	V					
rmmovq rA, D(rB)	4	0	rA	rB	D					
mrmmovq D(rB), rA	5	0	rA	rB	D					
OPq rA, rB	6	fn	rA	rB						
jXX Dest	7	fn	Dest							
call Dest	8	0	Dest							
ret	9	0								
pushq rA	A	0	rA	F						
popq rA	B	0	rA	F						



Y86-64 指令集

计算机组成原理

Byte	0	1	2	3	4	5	6	7	8	9	
halt	0	0							rrmovq	2 0	
nop	1	0							cmovle	2 1	
cmovXX rA, rB	2	fn	rA	rB	→ {					cmove	2 3
irmovq V, rB	3	0	F	rB	V					cmovne	2 4
rmmovq rA, D(rB)	4	0	rA	rB	D					cmovge	2 5
rrmovq D(rB), rA	5	0	rA	rB	D					cmovg	2 6
OPq rA, rB	6	fn	rA	rB							
jXX Dest	7	fn	Dest								
call Dest	8	0	Dest								
ret	9	0									
pushq rA	A	0	rA	F							
popq rA	B	0	rA	F							



Y86-64 指令集

计算机组成原理

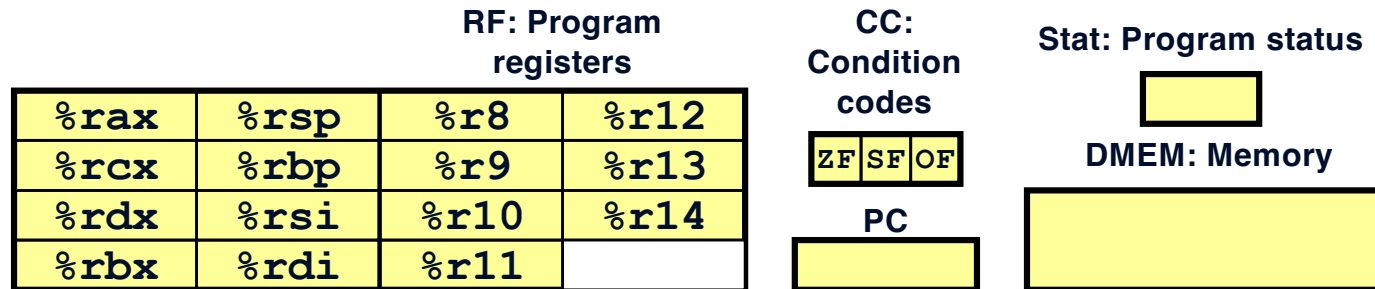
Byte	0	1	2	3	4	5	6	7		
									jmp	7 0
halt	0	0							jle	7 1
nop	1	0							j1	7 2
cmovXX rA, rB	2	fn	rA	rB					je	7 3
irmovq V, rB	3	0	F	rB	V				jne	7 4
rmmovq rA, D(rB)	4	0	rA	rB	D				jge	7 5
mrmovq D(rB), rA	5	0	rA	rB	D				jg	7 6
OPq rA, rB	6	fn	rA	rB						
jXX Dest	7	fn	Dest							
call Dest	8	0	Dest							
ret	9	0								
pushq rA	A	0	rA	F						
popq rA	B	0	rA	F						



Y86-64 指令集

计算机组成原理

Byte	0	1	2	3	4	5	6	7	8	9						
halt	0	0														
nop	1	0														
cmovXX rA, rB	2	fn	rA	rB												
irmovq V, rB	3	0	F	rB	V											
rmmovq rA, D(rB)	4	0	rA	rB	D											
mrmovq D(rB), rA	5	0	rA	rB	D											
OPq rA, rB	6	fn	rA	rB												
jXX Dest	7	fn	Dest													
call Dest	8	0	Dest													
ret	9	0														
pushq rA	A	0	rA	F												
popq rA	B	0	rA	F												



- Program Registers
 - 15 registers (omit %r15). Each 64 bits
- Condition Codes
 - Single-bit flags set by arithmetic or logical instructions
 - ZF: Zero SF: Negative OF: Overflow
- Program Counter
 - Indicates address of next instruction
- Program Status
 - Indicates either normal operation or some error condition
- Memory
 - Byte-addressable storage array
 - Words stored in little-endian byte order



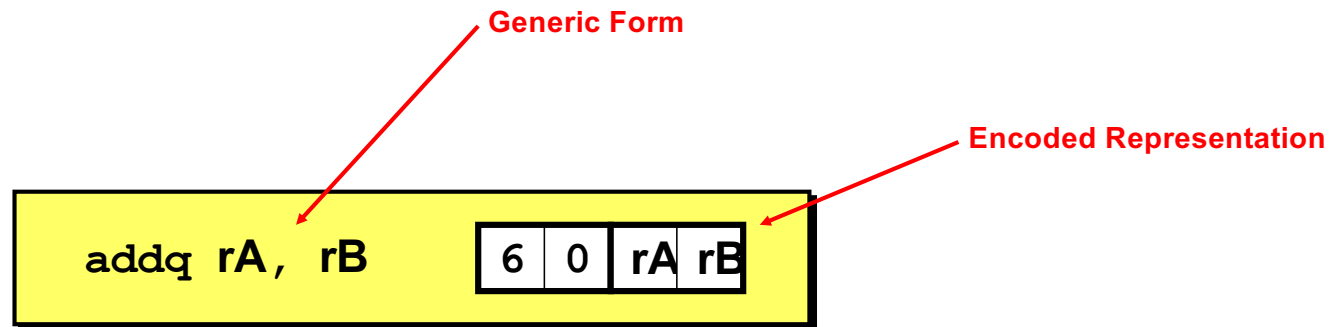
- Each register has 4-bit ID

%rax	0	%r8	8
%rcx	1	%r9	9
%rdx	2	%r10	A
%rbx	3	%r11	B
%rsp	4	%r12	C
%rbp	5	%r13	D
%rsi	6	%r14	E
%rdi	7	No Register	F

- Same encoding as in x86-64
- Register ID 15 (0xF) indicates “no register”
 - Will use this in our hardware design in multiple places



• Addition Instruction



- Add value in register rA to that in register rB
 - Store result in register rB
 - Note that Y86-64 only allows addition to be applied to register data
- Set condition codes based on result
- e.g., **addq %rax,%rsi** Encoding: **60 06**
- Two-byte encoding
 - First indicates instruction type
 - Second gives source and destination registers



运算逻辑指令

Instruction Code

Add

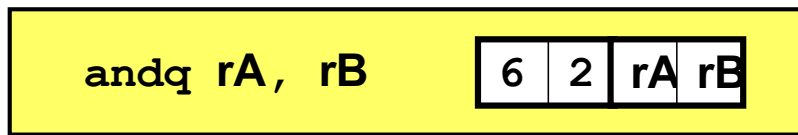


Function Code

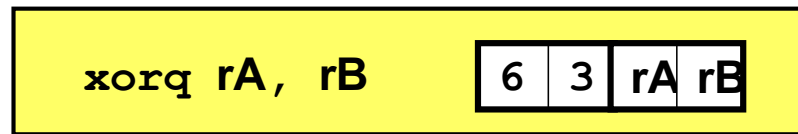
Subtract (rA from rB)



And



Exclusive-Or



- Refer to generically as “OPq”
- Encodings differ only by “function code”
 - Low-order 4 bytes in first instruction word
- Set condition codes as side effect



传送指令

Register → Register

`rrmovq rA, rB`



Immediate → Register

`irmovq V, rB`



Register → Memory

`rmmovq rA, D(rB)`



Memory → Register

`mrmovq D(rB), rA`



- Like the x86-64 `movq` instruction
- Simpler format for memory addresses
- Give different names to keep them distinct



Move 指令举例

X86-64

```
movq $0xabcd, %rdx
```

Encoding: 30 f2 cd ab 00 00 00 00 00 00

```
movq %rsp, %rbx
```

Encoding: 20 43

```
movq -12(%rbp), %rcx
```

Encoding: 50 15 f4 ff ff ff ff ff ff ff

```
movq %rsi, 0x41c(%rsp)
```

Encoding: 40 64 1c 04 00 00 00 00 00 00

Y86-64

```
irmovq $0xabcd, %rdx
```

```
rrmovq %rsp, %rbx
```

```
mrmovq -12(%rbp), %rcx
```

```
rmmovq %rsi, 0x41c(%rsp)
```



条件传送指令

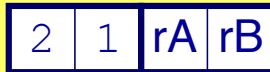
Move Unconditionally

`rrmovq rA, rB`



Move When Less or Equal

`cmovle rA, rB`



Move When Less

`cmovl rA, rB`



Move When Equal

`cmove rA, rB`



Move When Not Equal

`cmovne rA, rB`



Move When Greater or Equal

`cmovge rA, rB`



Move When Greater

`cmovg rA, rB`



- Refer to generically as “`cmovXX`”
- Encodings differ only by “function code”
- Based on values of condition codes
- Variants of `rrmovq` instruction
 - (Conditionally) copy value from source to destination register



跳转指令

Jump (Conditionally)



- Refer to generically as “j XX”
- Encodings differ only by “function code” fn
- Based on values of condition codes
- Same as x86-64 counterparts
- Encode full destination address
 - Unlike PC-relative addressing seen in x86-64



跳转指令

Jump Unconditionally

jmp Dest 7 0 Dest

Jump When Less or Equal

jle Dest 7 1 Dest

Jump When Less

j1 Dest 7 2 Dest

Jump When Equal

je Dest 7 3 Dest

Jump When Not Equal

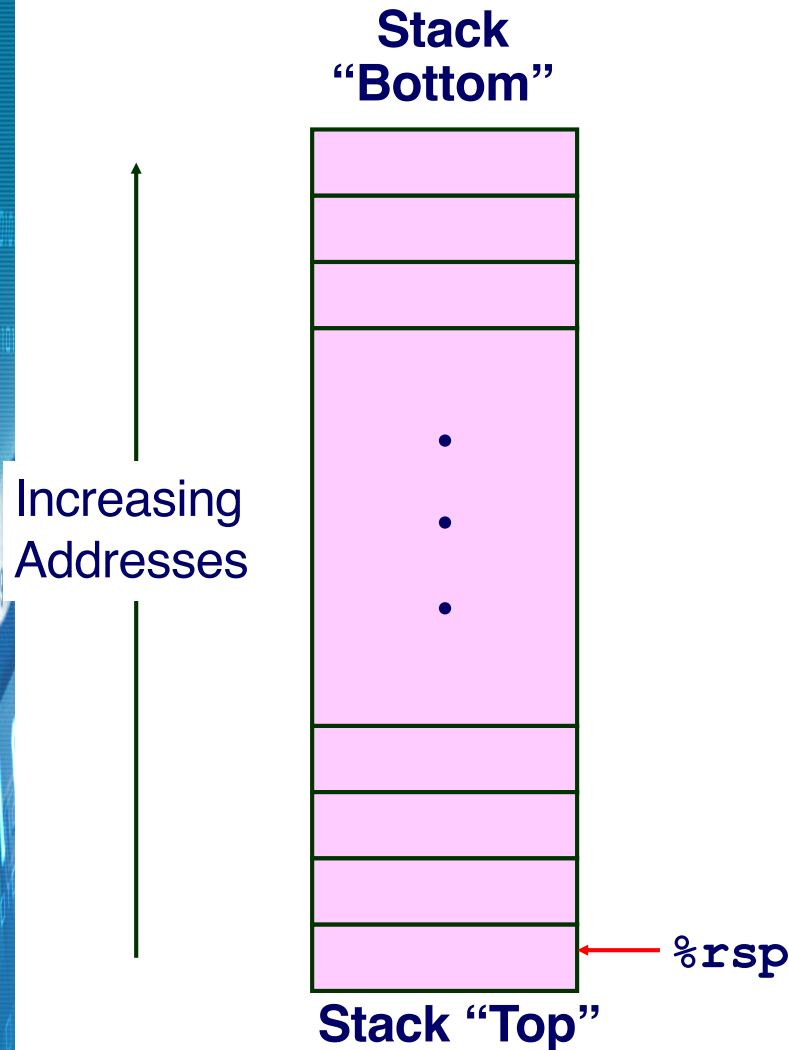
jne Dest 7 4 Dest

Jump When Greater or Equal

jge Dest 7 5 Dest

Jump When Greater

jg Dest 7 6 Dest



- Region of memory holding program data
- Used in Y86-64 (and x86-64) for supporting procedure calls
- Stack top indicated by `%rsp`
 - Address of top stack element
- Stack grows toward lower addresses
 - Top element is at lowest address in the stack
 - When pushing, must first decrement stack pointer
 - After popping, increment stack pointer

Stack Operations



Decrement $\%rsp$ by 8

Store word from rA to memory at $\%rsp$

Like x86-64



Read word from memory at $\%rsp$

Save in rA

Increment $\%rsp$ by 8

Like x86-64



- Push address of next instruction onto stack
- Start executing instructions at Dest
- Like x86-64



- Pop value from stack
- Use as address for next instruction
- Like x86-64



其他指令



- Don't do anything



- Stop executing instructions
- x86-64 has comparable instruction, but can't execute it in user mode
- We will use it to stop the simulator
- Encoding ensures that program hitting memory initialized to zero will halt



Y86-64异常

Mnemonic	Code
AOK	1

Mnemonic	Code
HLT	2

Mnemonic	Code
ADR	3

Mnemonic	Code
INS	4

- 1 Normal operation
- 2 Halt instruction encountered
- 3 Bad address (either instruction or data) encountered
- 4 Invalid instruction encountered

- Desired Behavior
 - If AOK, keep going
 - Otherwise, stop program execution



Y86-64 代码生成EXAMPLE

```
long len2(long *a)
{
    long ip = (long) a;
    long val = *(long *) ip;
    long len = 0;
    while (val) {
        ip += sizeof(long);
        len++;
        val = *(long *) ip;
    }
    return len;
}
```



Y86-64 代码Example

计算机组成原理

```
long len(long *a)
{
    long ip = (long) a;
    long val = *(long *) ip;
    long len = 0;
    while (val) {
        ip += sizeof(long);
        len++;
        val = *(long *) ip;
    }
    return len;}

```

Register	Use
%rdi	a
%rax	len
%rdx	val
%r8	1
%r9	8

```
len:
    irmovq $1, %r8          # Constant 1
    irmovq $8, %r9          # Constant 8
    irmovq $0, %rax         # len = 0
    mrmovq (%rdi), %rdx     # val = *a
    andq %rdx, %rdx         # Test val
    je Done                 # If zero, goto Done
Loop:
    addq %r8, %rax          # len++
    addq %r9, %rdi          # a++
    mrmovq (%rdi), %rdx     # val = *a
    andq %rdx, %rdx         # Test val
    jne Loop                # If !0, goto Loop
Done:
    ret

```



Y86-64 程序结构

计算机组成原理

```
init:                                # 初始化
    .pos 0                          # 汇编器伪指令，从地址0处产生代码
    irmovq stack %rsp              # 初始化栈指针
    call Main
    halt

    .align 8                        # 数据，8字节对齐的数组
array:
    . . .

Main:                                # Main function
    . . .
    call len    . . .

len:                                  # len function
    . . .

    .pos 0x200                      # 栈起始地址，向低增长，要防止覆盖代码和数据
stack:
```



Main:

```
irmovq array,%rdi  
# call len(array)  
call len  
ret
```

- Set up call to len
 - Follow x86-64 procedure conventions
 - Push array address as argument



```
unix> ./yas len.yo
```

- Generates “object code” file `len.yo`
 - Actually looks like disassembler output

```
0x054: | len:
0x054: 30f8010000000000000000 |   irmovq $1, %r8           # Constant 1
0x05e: 30f9080000000000000000 |   irmovq $8, %r9           # Constant 8
0x068: 30f0000000000000000000 |   irmovq $0, %rax          # len = 0
0x072: 5027000000000000000000 |   mrmovq (%rdi), %rdx       # val = *a
0x07c: 6222 |   andq %rdx, %rdx          # Test val
0x07e: 73a00000000000000000 |   je Done                  # If zero, goto Done
0x087: | Loop:
0x087: 6080 |   addq %r8, %rax           # len++
0x089: 6097 |   addq %r9, %rdi           # a++
0x08b: 5027000000000000000000 |   mrmovq (%rdi), %rdx       # val = *a
0x095: 6222 |   andq %rdx, %rdx          # Test val
0x097: 74870000000000000000 |   jne Loop                 # If !0, goto Loop
0x0a0: | Done:
0x0a0: 90 |   ret
```



```
unix> ./yis len.yo
```

– Instruction set simulator

- Computes effect of each instruction on processor state
- Prints changes in state from original

```
Stopped in 33 steps at PC = 0x13.  Status 'HLT', CC Z=1 S=0 O=0
```

```
Changes to registers:
```

%rax:	0x0000000000000000	0x0000000000000004
%rsp:	0x0000000000000000	0x0000000000000100
%rdi:	0x0000000000000000	0x0000000000000038
%r8:	0x0000000000000000	0x0000000000000001
%r9:	0x0000000000000000	0x0000000000000008

```
Changes to memory:
```

0x00f0:	0x0000000000000000	0x0000000000000053
0x00f8:	0x0000000000000000	0x0000000000000013